

A. Veredicto Executivo

A análise indica que **é tecnicamente possível** implementar o CalcMot no iOS, mas **com fortes limitações e riscos**. iOS não oferece nenhum equivalente público ao *AccessibilityService* do Android para ler a interface de outros apps; o único caminho viável hoje é capturar a tela do sistema via *ReplayKit* ou *ScreenCaptureKit*. Com consentimento explícito do usuário, é possível iniciar uma transmissão de tela em segundo plano (através de um **Broadcast Upload Extension** do *ReplayKit*), receber frames de vídeo do Uber/99/inDrive, aplicar OCR local e calcular as métricas em milissegundos. Frameworks oficiais da Apple (Vision, ActivityKit, etc.) atendem às necessidades, desde que o usuário dispare manualmente a gravação de tela e o app lide com as restrições de sandbox.

PROVADO: Apple confirma que não há API pública para inspecionar a interface de outros apps – nem um *AccessibilityService* de sistema (ou similar) [140†L57-L62] [109†L83-L93]. Em iOS 17+, *ReplayKit* será substituído por *ScreenCaptureKit* [140†L57-L62], mas isso não muda o fato de que só há essa abordagem de captura de tela. Outros métodos (XCTest, injeção de código, acessibilidade cruzada) são inviáveis ou proibidos.

EVIDÊNCIA FORTE: O *ReplayKit Broadcast Extension* é o único mecanismo suportado para captura de tela em segundo plano [109†L83-L93], mas exige que o usuário inicie a gravação (via um *RPSYSTEMBroadcastPickerView*). Não há como disparar isso programaticamente [57†L64-L73]. Fontes da Apple sugerem criar um protótipo para avaliar robustez (por exemplo, se a extensão continua ativa ao alternar apps [109†L83-L93]). Além disso, a interface de transmissão exibe uma barra vermelha de gravação na tela, o que pode gerar desconforto ao usuário [109†L117-L126].

EVIDÊNCIA FORTE: Já existem apps na App Store que fazem gravação de tela local (por exemplo, “Screen Recorder Voice & Video”), indicando que capturar tela on-device com consentimento é permitido [123†L61-L69]. Esses apps enfatizam que todo processamento é offline, o que alinha com as diretrizes de privacidade (aplicativo não armazena imagens nem envia dados a terceiros [123†L61-L69]). Portanto, embora a Apple não proíba explicitamente a captura de tela in-app autorizada pelo usuário, o caso é “cinza” – sensível, mas potencialmente aprovável se implementado conforme as regras (uso de APIs públicas e foco em privacidade local).

INFERIDO: Dada a falta de overlay nativo no iOS, a apresentação dos resultados exigirá criatividade (por exemplo, Live Activities/Dynamic Island). Essas soluções são públicas e documentadas (ActivityKit), mas não oferecem a mesma latência imediata de um overlay Android. É preciso aceitar trade-offs de UX.

Conclusão: O CalcMot no iPhone **é viável em teoria**, usando *ReplayKit/ScreenCaptureKit* + *Vision OCR* + *ActivityKit*. No entanto, trata-se de uma engenharia complexa. O maior bloqueio técnico é justamente a captura de tela de outro app (somente via Broadcast Extension com consentimento). O maior risco para App Store é o contexto de privacidade – mesmo que on-device e autorizado, o app “espia” outra aplicação, o que exige explicitação clara ao usuário (e melhorias contínuas se a política mudar).

B. Matriz de Capacidades

Capacidade	Android atual	iOS possível	Tecnologia	Condições
Leitura de UI de outro app	AccessibilityService	Não suportado (só dentro do próprio app)	UIAccessibility (somente interno)	PRO
Captura de tela / streaming	MediaProjection API	<i>ReplayKit</i> (Broadcast) / <i>ScreenCaptureKit</i>	RPBroadcastSampleHandler, RPScreenRecorder, SCStream (iOS17+) 【140†L57-L62】	PRO
OCR de texto	ML Kit / Tesseract	<i>Vision</i> VNRecognizeTextRequest	Vision Framework (recognitionLevel etc.)	PRO
Execução em background	Acessibilidade sempre ativo	Broadcast Extension (tempo limitado/ indeterminado)	RPBroadcastSampleHandler (possível <protótipo>) 【109†L83-L93】	EVID FOR
Detector de ofertas na tela	Heurísticas (view tree + OCR)	Heurísticas de imagem (hash/diff) + OCR	CVPixelBuffer diff, Core ML ou Vision	DED
Overlays (UI)	WindowManager Overlay	Não nativo – usar Live Activities / notificações	UIKit, Local Notification, AVSpeech, Haptics	DED
Live Activity / Dynamic Island	–	<i>Sim</i> (iOS 16.1+)	UIKit (Lock Screen / Dynamic Island)	PRO
Comunicação App ↔ Extensão	Bound service / eventos	App Group (UserDefaults / arquivos) + Darwin Notifications 【106†L24-L32】	CFNotificationCenterGetDarwinNotifyCenter	PRO
Áudio (voz)	Text-to-Speech / Media	AVSpeechSynthesizer (áudio local)	AVFoundation / Speech Synthesis	PRO
Haptics (vibração)	Vibrator API	UIFeedbackGenerator (Feedback)	UIKit (UIImpactFeedbackGenerator)	PRO

C. Arquitetura – Uber

- **Início de sessão:** O motorista abre o CalcMot e toca em “Iniciar sessão”. Um `RPSystemBroadcastPickerView` é exibido para o usuário ativar a transmissão de tela **【57†L64-L73】**. O sistema do iPhone solicita permissão e mostra o widget de seleção (ligado à nossa *Broadcast Upload Extension*). Após o usuário confirmar, inicia-se a gravação (barra vermelha de status aparece).
- **Captura de tela:** O *Broadcast Upload Extension* (`RPBroadcastSampleHandler`) entra em ação. Ele recebe callbacks periódicos com `CMSampleBuffer` do tipo vídeo (com resolução cheia da tela). Cada amostra pode ser convertida em `CVPixelBuffer` para análise.

- **Detecção de oferta:** O extension executa uma lógica simples de detecção. Por exemplo, monitora variações na região da tela onde o preço/aplicativo Uber aparece (usando hash de imagem, diferença de quadros ou detecção de UI estática). Assim, ao detectar uma nova oferta (mudança significativa no conteúdo), procede para o próximo passo.
- **OCR local:** O `CVPixelBuffer` da região de interesse é passado a uma requisição `VNRecognizeTextRequest`. Configuramos `recognitionLevel = .fast` e `usesLanguageCorrection = false` para velocidade. Definimos `regionOfInterest` cobrindo onde “R\$ x,xx”, distância e tempo aparecem, e adicionamos `customWords = ["R$", "km", "min"]`. O framework Vision devolve as strings detectadas (por exemplo `["R$ 12,50", "3,4 km", "7 min"]`).
- **Parsing e normalização:** Essas strings são repassadas a um *parser* específico do Uber (implementado no módulo comum em Kotlin), que extrai valores numéricos e constrói um objeto `Offer(platform=UBER, price=12.50, tripDistanceKm=3.4, tripDurationMinutes=7, ...)`. O objeto é enviado ao **núcleo de cálculo CalcMot** (compartilhado no KMP) que calcula R\$/km, R\$/hora e retorna um `OfferAnalysis` (classificação “boa”, “média” ou “ruim”).
- **Apresentação do resultado:** O resultado da análise é comunicado de volta ao motorista. Como não há overlay livre em iOS, usamos canais permitidos. Exemplo: atualizamos uma *Live Activity*/Dynamic Island ativa para mostrar a classificação e métricas. Alternativamente ou adicionalmente, enviamos uma notificação local (banner) e/ou ativamos áudio/haptics. Todo dado transitório é armazenado em App Group (UserDefaults) e sinalizado via Darwin Notification para o app principal atualizar a interface (ou para o próprio *Live Activity* via ActivityKit).

Fluxo resumido (Mermaid):

```
graph LR
    subgraph Apps_in_foreground [Apps em foreground]
        UberApp[Uber\ App]
    end
    subgraph CalcMot
        direction TB
        UI[UserUI[App CalcMot - Inicio]] --> BCP[BroadcastPicker[RPSystemBroadcastPickerView]]
        BCP -->|Usuário inicia| BExt[BroadcastExt[Broadcast Upload Extension]]
        BExt --> VC[VisualChange[Detecção de mudança visual]]
        VC --> OCR[OCR[Vision OCR (VNRecognizeText)]]
        OCR --> PU[ParserUber[Parser Uber (KMP)]]
        PU --> N[Normalize[Offer Normalizado]]
        N --> CE[CalcEngine[CalcMotEngine (KMP)]]
        CE --> P[Presentation[Live Activity / Notificação / Áudio]]
    end
    UberApp -->|frame de tela| BExt
```

D. Arquitetura – 99

A arquitetura para a 99 é *praticamente idêntica*. Como não existe API pública da 99, seguimos pipeline de captura+OCR:

1. **Broadcast Starter:** Usuário aciona a gravação de tela do CalcMot (via `RPSystemBroadcastPickerView`).
2. **ReplayKit Extension:** Recebe frames mesmo com a 99 em primeiro plano.
3. **Detecção de oferta:** Observa a tela da 99 (preço/km/min) por mudanças.
4. **OCR:** `VNRecognizeTextRequest` é aplicado na região relevante (preço e distância da corrida exibidos).
5. **Parser 99:** Um parser específico extrai `price`, `distance`, `duration`, etc.
6. **Cálculo:** O domínio compartilhado normaliza em `Offer` e faz os cálculos.
7. **Apresentação:** Exibe resultado via Live Activity/Notificação/feedback.

Não há sobreposição direta da interface da 99; todos os dados são extraídos via imagem. A velocidade de detecção e OCR precisa ser alta, pois as ofertas da 99 duram poucos segundos, similar ao Uber.

E. Arquitetura – inDrive

O pipeline do inDrive segue o mesmo padrão das outras duas plataformas:

1. O motorista inicia sessão e ativa a transmissão de tela.
2. O *Broadcast Upload Extension* captura os frames da inDrive em tempo real.
3. Detectamos o surgimento de uma nova oferta (mudança visual na UI).
4. Aplicamos OCR (Vision) nos campos de preço/km/min da interface do inDrive.
5. Um parser dedicado do inDrive analisa o texto reconhecido e constrói um `Offer`.
6. Cálculos de rentabilidade são feitos pelo núcleo comum.
7. O resultado é exibido via Live Activity/Dynamic Island ou outro canal alternativo (notificações, feedback etc.).

Até onde se sabe, a interface do inDrive é diferente, mas também baseada em texto claro, tornando viável a extração via OCR. Cada app (Uber/99/inDrive) pode requerer ajustes finos de OCR e parsing, mas a arquitetura de alto nível permanece a mesma.

F. Arquitetura Alvo iOS (Mermaid)

A seguir, o diagrama de componentes gerais para iOS:

```
graph TD
    subgraph Smartphone
        User[Usuário] --> CalcMotApp[CalcMot App (foreground)]
        CalcMotApp --> BroadcastPicker[RPSystemBroadcastPickerView]
        BroadcastPicker -->|Usuário inicia| BroadcastExt[Broadcast Upload Extension]
        subgraph CaptureExt
            BroadcastExt -->|CMSampleBuffer| FrameProc[Processador de Frame]
            FrameProc --> DiffDetect[Detector de Mudança Visual]
        end
    end
```


L Platform/
chamando shared lib)

Adapters Swift ↔ Kotlin (por exemplo,

- **shared/commonMain**: inclui `data class Offer`, lógica de cálculos (`CalcMotEngine`), enums e regras (por exemplo, thresholds do semáforo). Também definimos interfaces (ou `expect` functions) para OCR, parser e fonte de ofertas.
- **androidMain**: implementa `OfferSource` usando `AccessibilityService` e ML Kit, se houver. (No caso do Android existir, mas aqui focamos em iOS.)
- **iosMain**: implementa as interfaces para iOS, por exemplo usando Vision para `TextRecognizer` ou adaptando `OfferSource` para o Broadcast Extension.
- **iosApp**: contém a UI em Swift/SwiftUI, a configuração do `BroadcastPicker`, handlers do `ActivityKit`, etc. Ele consome o módulo compartilhado compilado como framework iOS (via Kotlin/Native).

Esse design maximiza o reuso do código de domínio e de parsing (principal peso do KMP), enquanto mantém lógica específica de plataforma nas camadas adequadas.

H. Matriz de Reaproveitamento de Código

Componente Android atual	Compartilhável?	Tecnologias	Alterações Necessárias
Modelos de domínio	Sim	KMP (commonMain)	Mover <code>Offer</code> , <code>OfferAnalysis</code> , <code>Classification</code> para commonMain. Pequenas mudanças nos tipos (usar <code>Double?</code> em vez de primitivos Java).
Cálculos R\$/km, R\$/h, semáforo	Sim	KMP (commonMain)	Reusar diretamente; testes existentes servem.
Regras de negócio / limites	Sim	KMP	Mover thresholds para commonMain. Ajustar se necessário (pode extrair para configuração).
Parser Uber	Parcialmente	KMP (commonMain)	Reescrever para não depender de Views Android. Em commonMain, trabalhar com lista de strings; iOS fornece texto OCR.
Parser 99	Parcialmente	KMP	Similar ao Uber: lógica de parsing compartilhável, mas extração de texto por OCR será no iOS.
AccessibilityService	Não	-	Permanecerá no Android (não reaproveitável).
OCR (Android)	Não (API-gráficas)	-	iOS usará Vision (não reutilizável). No shared, interface para OCR se desejado.

Componente Android atual	Compartilhável?	Tecnologias	Alterações Necessárias
Overlay Android	Não	-	iOS utilizará Live Activity / notificações (nova implementação).
Log/Analytics	Parcialmente	KMP	Contratos de eventos (classes/enum) podem ser compartilhados; implementação de envio é plataforma.
Armazenamento	Parcialmente	KMP + Platform	Estruturas de dados (p.ex. OfferId) podem ser comuns; acesso SQLite ou DB fica em cada app.

Em resumo, **os cálculos e as regras de domain podem ser quase 100% compartilhados** via KMP. Os *parsers* de texto (transformar strings em números) também podem viver em commonMain, bastando receber do iOS strings extraídos. Já a captura de tela, OCR, e UI serão totalmente reescritos em Swift.

I. Estrutura de Repositório

Uma estrutura de projeto possível:

```
calcMot/
├── shared/                                # Módulo Kotlin Multiplatform
│   ├── commonMain/
│   ├── androidMain/
│   └── iosMain/
├── androidApp/                            # Código específico do app Android atual
└── iosApp/                                # Projeto Xcode para iOS
    ├── CalcMotApp.xcodeproj
    ├── App/                              # Código Swift/SwiftUI da UI principal
    │   ├── ContentView.swift
    │   ├── ActivityHandlers.swift
    │   └── ...
    ├── Extensions/                       # Targets de extensão
    │   ├── ScreenBroadcast/              # Broadcast Upload Extension
    │   │   ├── SampleHandler.swift
    │   │   └── ...
    ├── Platform/                         # Código de ponte Swift ↔ Kotlin
    │   └── CalcMotShared.kt (gerado)
    └── Resources/
```

- Em **shared**, usamos Gradle/Kotlin para gerar uma *framework* iOS (`calcMot.framework`) que o projeto Xcode incluirá (via CocoaPods ou pacote XCFramework).
- **iOS App** (iosApp): target principal com SwiftUI, gerencia Live Activities e fornece o `RPSystemBroadcastPickerView`.

- **ScreenBroadcast**: target de extensão (Broadcast Upload Extension) onde processamos `CMSampleBuffer` e rodamos o OCR. Ele compartilha o container App Group do `iosApp` para comunicação.
- **Platform**: pasta onde colocamos eventuais adaptadores ou inicializações (por exemplo, registrar notificações do Kotlin).

J. Interfaces Críticas

Algumas interfaces e contratos fundamentais (em Kotlin para o shared):

```
// Módulos compartilhados (commonMain)

enum class Platform { UBER, NINETY_NINE, INDRIVE }

data class Offer(
    val platform: Platform,
    val price: Double,
    val tripDistanceKm: Double?,
    val pickupDistanceKm: Double?,
    val tripDurationMinutes: Int?,
    val pickupDurationMinutes: Int?,
    val rating: Double?
)

data class OfferAnalysis(
    val pricePerKm: Double?,
    val pricePerHour: Double?,
    val classification: Classification
)

enum class Classification { GOOD, AVERAGE, POOR }

// Interface para reconhecimento de texto (OCR)
interface TextRecognizer {
    suspend fun recognize(frame: CVPixelBuffer): List<String>
}

// Fonte de ofertas detectadas ( fluxo de ofertas extraídas da UI )
interface OfferSource {
    /**
     * Emite ofertas assim que são detectadas na tela.
     * Pode ser usado com Kotlin Flow para notificações em tempo real.
     */
    fun offers(): Flow<Offer>
}
```

No lado **iOS (Swift)**, usaríamos a framework KMP gerada (`CalcMotShared.framework`). Por exemplo, chamaríamos:

```

import CalcMotShared

// Criar um Offer (Kotlin data class) para análise:
let price = 12.50
let distance = 3.4
let offer = CalcMotSharedOffer(
    platform: CalcMotSharedPlatform.uber,
    price: price,
    tripDistanceKm: distance,
    pickupDistanceKm: nil,
    tripDurationMinutes: 7,
    pickupDurationMinutes: nil,
    rating: nil
)

// Analisar oferta no núcleo compartilhado:
let engine = CalcMotSharedCalcMotEngine()
let analysis = engine.analyzeOffer(offer: offer) // OfferAnalysis retornado

print("R$/km: \(analysis.pricePerKm), classificação: \(
analysis.classification)")

```

Aqui, `CalcMotSharedCalcMotEngine` e outras classes são traduzidas pelo Kotlin/Native para uso em Swift. Os métodos e propriedades seguem a convenção Kotlin/Native, e tratamos nullability conforme as APIs geradas.

K. Engenharia ReplayKit

Fluxo ReplayKit: iOS oferece `RPScreenRecorder` e extensões de broadcast. Para captura cross-app, usamos o *Broadcast Upload Extension*:

- **RPSystemBroadcastPickerView:** View que embute um botão de gravação. Configuramos seu `preferredExtension` com o Bundle ID do nosso Broadcast Extension. Quando o usuário toca, o sistema exibe uma folha para selecionar nosso serviço de broadcast. O usuário deve confirmar o início da gravação; **não há modo programático** de iniciar isso [57†L64-L73] .
- **RPBroadcastSampleHandler:** Classe no target da extensão. O sistema chama:
 - `broadcastStarted(withSetupInfo:)` quando a transmissão começa.
 - `processSampleBuffer(_:with:)` para cada novo frame de vídeo (tipo `.video`) ou áudio (tipo `.audioApp` / `.audioMic`). Recebemos objetos `CMSampleBuffer`.
 - `broadcastFinished()` ou `processSampleBuffer(... .video, .stopped)` quando acaba.
- **Formato dos frames:** Cada `CMSampleBuffer` de vídeo contém um `CVPixelBuffer` (a imagem da tela). A taxa de quadros (**FPS**) corresponde à do dispositivo (tipicamente 30–60 FPS). Em iPhones modernos, poderá chegar a 60fps em 1080p. As amostras vêm em tempo real, com latências do sistema. Testes (ScreenCaptureKit no macOS) indicam latência inicial de ~30–100ms

para 1080p [138†L640-L643] . A extensão deve extrair o pixelBuffer para processar com o Vision.

- **Limites:** A extensão de broadcast corre em background enquanto o usuário estiver transmitindo. A Apple não documenta um limite fixo, mas recomenda-se testar se ela permanece ativa após horas de uso [109†L83-L93] . O sistema mostrará continuamente uma barra/ponto vermelho indicando gravação, e o usuário pode parar a qualquer momento. Em uso intenso, a extensão pode ser encerrada pelo iOS (jetsam) se exceder memória/CPU. **Cuidado:** não devemos carregar a CPU sem necessidade. O sample handler roda em background, mas não é garantido indefinidamente. Apple sugere *protótipo* para avaliar estabilidade [109†L83-L93] .
- **Desempenho:** A captura de tela puro é surpreendentemente leve. Em hardware Apple Silicon (próximos aos chips de iPhone), captura full 60fps de vídeo 1080p consome apenas $\approx 1.9\%$ de um núcleo [138†L635-L638] . A tabela de benchmarks mostra $\sim 30\text{--}100\text{ms}$ de latência no primeiro quadro 1080p [138†L640-L643] . Portanto, a sobrecarga de captura é baixa; o maior custo estará no OCR subsequente e no processamento.
- **Direitos e entitlements:** A implementação exige:
 - Um *Broadcast Upload Extension* configurado no Xcode.
 - No Info.plist do app: uma string de uso de gravação de tela (por exemplo `NSScreenCaptureUsageDescription`), caso o iOS 17+ peça permissão (nosso app é o originador da requisição).
 - O usuário vê consentimento explícito de “Gravação de Tela” e que dados ficam locais.
- **Fluxos de controle:** Em resumo:
 - App exibe `RPSystemBroadcastPickerView` . Usuário toca e seleciona nossa extensão.
 - O sistema pede permissão (“CalcMot quer gravar a tela?”).
 - Com permissão, a extensão começa e recebe frames da tela (também de aplicativos em foreground).
 - Enquanto ativa, aparece indicador de gravação e o app de destino (Uber/99/inDrive) continua normal. A extensão processa frames em background.

L. Engenharia OCR

Usaremos o **Vision Framework** da Apple para OCR em tempo real:

- **VNRecognizeTextRequest:** A requisição básica do Vision para reconhecer texto em imagem. Ajustamos parâmetros para alto desempenho:
- `recognitionLevel = .fast` (menos preciso que `.accurate` , mas muito mais rápido).
- `recognitionLanguages = ["pt-BR", "es-ES", "en-US"]` (incluímos línguas relevantes e símbolos).
- `usesLanguageCorrection = false` (desativa correção ortográfica, pois trabalhamos com números e siglas).
- `minimumTextHeight = 0.02` (reconhecer apenas textos de altura relativa maior, para ignorar itens menores).
- `customWords = ["R$", "km", "min"]` (garantir reconhecimento de símbolos-chave).

- **ROI:** Muito importante é restringir a área de busca. Configuramos `request.regionOfInterest` para a região normalizada da tela onde aparecem preço, distância e tempo da oferta. Por exemplo, se no Uber o preço fica no topo, colocamos algo como `{x:0.1, y:0.1, width:0.8, height:0.2}`. Isso limita drasticamente o processamento e acelera o reconhecimento.
- **Performance:** Em iPhones modernos (A15/A16), cada chamada de OCR em uma pequena ROI deve levar dezenas de milissegundos. Em benches, por exemplo, Vision podendo usar GPU ou Neural Engine, o custo costuma ficar abaixo de 100ms por frame de texto simples. (Não há dado público exato; estimamos <50ms por requisição em dispositivos atuais e talvez ~100ms em iPhone 11). Como procesamos frame-by-frame, devemos pular vários quadros: por exemplo, 30fps → podemos fazer OCR talvez a cada 1–2 frames úteis (ou quando detectar mudança). Técnicas de *frame skipping* ou hashing de imagem são recomendadas para evitar OCR redundante.
- **Parser especializado:** Após o OCR, teremos strings (ex: "R\$ 12,50", "3,4 km", "7 min"). O parser converte esses textos em dados numéricos (`Double` ou `Int`). Exemplo: remove "R\$", usa `Double.parse`, interpreta "min" e "km". Isso é trivial e implementado no módulo compartilhado (common). Cada app terá seu próprio parser para lidar com peculiaridades de formatação (ponto decimal, vírgula, etc.).
- **Detecção de oferta:** Não disparamos OCR o tempo inteiro. Uma heurística eficiente é comparar hashes visuais do frame atual com o anterior: se pouco mudou, pulamos OCR. Se detectamos mudança significativa no region-of-interest (por exemplo, diferença acima de um limiar), então executamos OCR. Outra estratégia: rodar OCR contínuo e ignorar se não aparecer "R\$", mas isso é menos eficiente. Como resultado, o pipeline ideal é algo como: *60 fps entrada* → **detector de mudança** → 1 frame relevante → ROI → OCR → Parser → Cálculo.

M. Estratégia de Apresentação

Sem um overlay direto, usamos meios permitidos para reportar resultados:

- **Live Activities / Dynamic Island:** É a solução mais próxima de um "overlay persistente". Criamos uma Live Activity (iOS 16.1+) que exibe no Lock Screen (tela bloqueada) e na Dynamic Island (iPhones Pro recentes). Mesmo com outro app em primeiro plano, a ilha dinâmica pode mostrar ícone ou número, permitindo atualizações frequentes [140†L57-L62]. A atualização do conteúdo da Activity pode ser feita pelo nosso app principal lendo dados compartilhados pela extensão (ex. via BackgroundTask ou notifications internas) e empurrando para o ActivityKit. *Latência:* em testes, Live Activities podem atualizar em poucos segundos, suficiente para decisões de corrida. *App Store:* ActivityKit é API pública e encorajada.
- **Notificações locais (banner):** Enviar uma notificação curta quando a análise ficar pronta. Isto aparece como banner no topo, mesmo com outro app ativo. Fornece feedback imediato ("Boa corrida! R\$30,00 – Ver detalhes"). Limitação: não é persistente e interrompe a tela momentaneamente. Útil como fallback caso Live Activity não esteja disponível (ex. iPhones antigos sem ilha dinâmica).
- **Haptics/Áudio:** Como reforço, podemos usar vibração ou sintetizador de voz. Exemplos: ao determinar "boa corrida", gerar um leve padrão de vibração (`UIImpactFeedbackGenerator`). Ou

falar “corrida boa”. Isso não distrai tanto quanto olhar para tela. *Limitação*: poucos dados podem ser transmitidos – só qualidade/cor do semáforo, sem detalhes. É permitido (p.ex. AVSpeechSynthesizer). Pode ser complemento, mas não substitui interface.

- **Picture-in-Picture (PiP)**: Embora tecnicamente viável (rodar um vídeo invisível em PiP para ter overlay), o uso do PiP fora de conteúdo de vídeo é *uma gambiarra proibida*. Apple proíbe usar PiP para propósitos de overlay fixo. Devemos **não usar PiP em produção**. Mencionamos apenas para pesquisa de alternativas não oficiais.

Resumo dos prós/contras:

- *Live Activities*: Funciona com app em background, baixa latência visual, API pública. Mas disponível apenas em iOS 16.1+ (todas as iPhones recentes) e limitado a UI restrita (texto simples). Rápido de atualizar via ActivityKit ou notificações push internas.
- *Notificações locais*: Simples e compatível com todas as versões, mas breves e intrusivas.
- *Áudio/Haptics*: Permite informação mesmo sem olhar, mas transmite só classificação geral. Poucos apps de direção usam voz para análises rápidas (mas é legal explorar, e App Store permite, já que acessibilidade no carro não é proibida).
- *Widget ou lockscreen*: Widgets comuns não são dinâmicos o suficiente; Lock Screen estático só via Live Activity.
- *Testes Internos*: Sugerimos testar as combinações para ver o que prende mais atenção do motorista sem distrair.

N. Orçamento de Performance

Construímos um orçamento de latência estimado (tempo médio por etapa) em um dispositivo recente (ex: iPhone 14) vs. um antigo (iPhone 11). Valores são aproximados e dependem de otimizações:

Etapa	Tempo (ideal, no-device rápido)	Tempo (aceitável, old-device)	Objetivo / Meta
Detecção de oferta	5–10 ms (hash ou diff de frame)	20–50 ms	< 50 ms
Recorte de ROI	1–2 ms	5–10 ms	~5 ms
OCR Vision	30–50 ms (1 requisição)	100–150 ms	< 100 ms
Parser (texto → números)	1–2 ms	2–5 ms	~2 ms
Cálculo e classificação	1–5 ms	5–10 ms	~5 ms
Comunicação interna	2–5 ms	5–10 ms	~5 ms
Apresentação/ atualização UI	50–100 ms (Live Activity update)	100–300 ms	< 200 ms

Etapa	Tempo (ideal, no-device rápido)	Tempo (aceitável, old-device)	Objetivo / Meta
TOTAL	~100 ms	~300–500 ms	Ideal <200 ms

- **Deteção de oferta:** Usando um hash leve ou diff de imagem, deveríamos gastar poucos milissegundos por frame (dominados pela cópia em memória). Mesmo em devices antigos, <50ms é plausível.
- **OCR Vision:** Em dispositivos recentes, uma requisição de OCR limitada à região de interesse (~20% da tela) leva poucas dezenas de milissegundos. Em iPhone 11/A13, pode chegar a ~150ms. Podemos reduzir esse custo limitando a frequência (p.ex. rodar OCR 5–10 vezes por segundo, ao invés de 60).
- **Parse e cálculo:** Praticamente instantâneo (sub-milissegundo). Não é gargalo.
- **Comunicação:** Escrever em UserDefaults ou arquivo e notificar via Darwin é rápido (milissegundos) se bem implementado em background.
- **Atualização de UI:** Criar/atualizar uma Live Activity ou enviar notificação local envolve algum overhead (p.ex. 100ms). Em geral, podemos exibir o resultado em <200ms após obtê-lo.

Metas: O objetivo é manter todo o fluxo abaixo de cerca de 500ms em aparelhos antigos, e idealmente ~100ms em aparelhos novos. O tempo total real dependerá de quanto pulamos quadros: por exemplo, processar apenas 1 em cada 6 frames dá margem para OCR demorado. Em tempo real, consideramos **<1s total** aceitável (motorista muitas vezes leva ~5–10s para decidir), mas ideal seria **<300ms** para não perder ofertas rápidas.

O. Viabilidade App Store

Cada abordagem proposta foi avaliada quanto ao uso de APIs públicas e às diretrizes da App Store:

- **Broadcast + On-Device OCR:** *Técnicamente possível.* Usa APIs públicas de ReplayKit/ScreenCaptureKit e Vision. Documentação da Apple apoia transmissão de tela on-device. Há apps aprovados que fazem gravação de tela local 【123†L61-L69】 (privacidade local garantida). *Conformidade:* Precisa de explicitação de permissão (política de privacidade clara) e usar consentimento do usuário. **Risco moderado:** embora permitido, revisores podem questionar “por que o app precisa gravar outra app?”. Referências oficiais não banem explicitamente (ao contrário de injeção de UI).
- **Classificação:** *TECNICAMENTE POSSÍVEL; API PÚBLICA; compatível via consentimento. Zona de risco moderado.*
- **Acessibilidade/Testes:** *Inviável.* Não há API pública para ler a interface de outro app em iOS. Qualquer tentativa (testes UI usando XCTest ou códigos privados) viola as regras de execução de App Store.
- **Classificação:** *INVIÁVEL; API não pública; proibido.*
- **Atalhos/Siri Onscreen:** *Técnicamente futura.* Não há hoje. Rumores de “Siri On-Screen Awareness” (futuro iOS 18) sugerem que apenas Siri poderá ler a tela. Sem nada atual.
- **Classificação:** *DESCONHECIDO; se disponível, seria API pública. Atualmente não existe solução.*

- **Captura por compartilhamento de screenshot (extensão de compartilhamento):** *Possível mas manual.* O usuário poderia, dentro do app Uber/99, tirar screenshot e usar “Compartilhar” para enviar ao CalcMot (com uma share extension). Aí sim o app poderia ler o screenshot. Isso cumpre diretrizes (o usuário explicitamente compartilha). Contudo, não cumpre o requisito de **tempo real**, pois depende da ação manual.
- **Classificação:** *POSSÍVEL (API Pública de extensão de compartilhamento); muito seguro para App Store; mas quebra o fluxo de tempo real.*
- **Picture-in-Picture (hacks):** *Não recomendado.* Embora algumas soluções da comunidade abusem do PiP para simular sobreposição, Apple expressamente desaprova tal uso. Usar PiP apenas para mostrar resultados seria claro abuso (Guideline 2.5.1/API reuso).
- **Classificação:** *ALTO RISCO/ proibido.* Pode até funcionar tecnicamente, mas certamente levaria a rejeição.
- **Live Activities + Dynamic Island:** *Compatível.* API pública (iOS 16.1+), aceita pela App Store. Não substitui transparência total, mas é o canal oficial para updates em lock screen/ilha. Requer que o app solicite permissão de notificações e inclua entitlements de ActivityKit. **Baixo risco.**

Em resumo, a **arquitetura recomendada** (Broadcast Ext + Vision + LiveActivity) usa apenas APIs públicas e documentadas. Não há garantia total da aprovação (cada revisão é única), mas ao enfatizar privacidade (processamento on-device, opt-in claro) e justificativa clara (auxílio ao motorista), as chances aumentam. Hackes ou APIs privadas devem ser evitadas a todo custo.

P. Arquitetura de Privacidade

Priorizamos **privacidade by design**:

- **Processamento local:** Todo OCR e cálculo acontecem no dispositivo, sem enviar qualquer frame ou texto ao servidor. O app obriga o uso *offline*. Como em **【123†L61-L69】**, ressaltamos no README/manifesto que nada é enviado para terceiros. Isso atende aos requisitos de LGPD (dados do passageiro/passageiro não são transmitidos nem armazenados).
- **Uso mínimo de dados:** A extensão imediatamente descarta o frame após o OCR – não armazenamos vídeos nem capturas. Guardamos apenas números e classificações (não sensíveis). Dados analíticos enviados ao backend (se houver) **não incluem** conteúdo pessoal; apenas métricas (“evento: boa_corrida exibida”).
- **Consentimento:** No `Info.plist` incluímos chaves de privacidade relevantes, por exemplo `NSScreenCaptureUsageDescription` (iOS 17+), explicando por que capturamos a tela. A UI inicial do app também deve explicar ao usuário (em próprio texto) que o uso de gravação de tela é voluntário e necessário para análise de corridas.
- **App Groups e criptografia:** Se usarmos UserDefaults compartilhado, ele fica criptografado pelo sandbox iOS. Não guardamos dados sensíveis além dos necessários.
- **Dados do passageiro:** Em oferta Uber/99/inDrive aparecem nomes ou localizações. Nossa arquitetura **não tenta extrair** esses campos. O foco são valores (R\$, km, min). Se por ventura outros textos forem reconhecidos, descartamos. Assim, não processamos nem armazenamos dados pessoais de terceiros.
- **Análítica e logs:** Qualquer dado de uso enviado a servidores (se houver) será anonimizado (sem conteúdo de tela), seguindo as regras de privacidade da App Store e LGPD.

Em síntese, mantemos tudo **on-device**, descartamos os frames, e pedimos apenas permissões absolutamente necessárias (gravação de tela). Isso deve satisfazer as diretrizes de privacidade da Apple e a legislação brasileira.

Q. Prova de Conceito (POC)

Para validar rapidamente a hipótese central (“É possível capturar uma oferta Uber em tempo real?”), criamos a menor POC possível:

1. **Projeto:** iOS Single View App (`CalcMotPOC`) + Broadcast Upload Extension (`CalcMotPOCExtension`). Sem UI além do `RPSystemBroadcastPickerView`.
2. **Passo a passo:**
3. No app principal, colocar um `RPSystemBroadcastPickerView` no storyboard. Configure seu `preferredExtension` para o ID da extensão.
4. Na extensão (`SampleHandler`), implemente:

```
override func broadcastStarted(withSetupInfo setupInfo: [String :
NSObject]?) { /* inicio */ }
override func processSampleBuffer(_ sampleBuffer: CMSampleBuffer, with
sampleBufferType: RPSampleBufferType) {
    guard sampleBufferType == .video,
        let pixelBuffer = CMSampleBufferGetImageBuffer(sampleBuffer)
    else { return }
    let request = VNRecognizeTextRequest { request, error in
        if let texts = request.results as?
[VNRecognizedTextObservation] {
            let recognized = texts.compactMap {
                $0.topCandidates(1).first?.string }
            print("OCR: \(recognized)")
        }
    }
    request.recognitionLevel = .fast
    let handler = VNImageRequestHandler(cvPixelBuffer: pixelBuffer,
options: [:])
    try? handler.perform([request])
}
override func broadcastFinished() { /* fim */ }
```

5. Execute em dispositivo: abra o CalcMotPOC, aperte o botão de broadcast, selecione a extensão, então abra o app Uber.
6. **Testes:** Confirmar que no console da extensão aparece o texto da oferta (preço, km, min) antes que ela desapareça. Ajustar `regionOfInterest` e `customWords` para melhorar o OCR se necessário.
7. **Critério de sucesso:** Obter no log algo como `["R$ 10,00", "2,5 km", "5 min"]`. Latência de OCR ~0.1–0.2s.

8. **Variantes mínimas:** Não inclui servidor, login, nem UI além do necessário. O objetivo é só testar captura+OCR. Se falhar (por exemplo, 0 frames chegam), interromper POC e ajustar.

R. Plano Experimental

Definimos experimentos isolados para validar premissas críticas. Cada EXP tem: hipótese, implementação mínima, métrica e critério de sucesso/aborto.

- **EXP-01: Frames ReplayKit** – *Hipótese:* A extensão recebe frames do Uber aberto.
 - *Implementação:* Use o POC acima, sem OCR; apenas conte frames (ou imprima um pixelBuffer). Coloque `print("Frame \(\timestamp)")` no sample handler.
 - *Métrica:* Recebimento de frames (número de logs por segundo).
 - *Sucesso:* Recebe frames de vídeo (ex. ≥ 10 fps).
 - *Abandono:* Se `processSampleBuffer` não é chamado quando o Uber estiver aberto, reprovar essa abordagem.
- **EXP-02: OCR Vision** – *Hipótese:* Vision consegue extrair preço/distância/tempo de uma imagem estática do Uber.
 - *Implementação:* Pegue um screenshot real do app Uber com oferta. Rodar `VNRecognizeTextRequest` numa MVVM de teste (fora de broadcast).
 - *Métrica:* Precisão do OCR (quão próximo do texto real; p.ex. erro ≤ 1 caractere em cada campo).
 - *Sucesso:* Detecta corretamente os campos principais em $\geq 90\%$ dos testes.
 - *Abandono:* Se Vision falhar consistentemente em reconhecer “R\$” e valores (mesmo ajustando customWords), precisar reavaliar (talvez testes de ML personalizados).
- **EXP-03: Latência OCR** – *Hipótese:* OCR + parse + cálculo ocorre em tempo suficiente.
 - *Implementação:* Dentro do EXP-01 ou POC do EXP-02, medir tempo de `handler.perform(...)` e de parse.
 - *Métrica:* Tempo total (ms) da rotina OCR→parser→resultado.
 - *Sucesso:* Latência total <500ms (idealmente <200ms) por oferta.
 - *Abandono:* Se >1s constantemente, considerar que é lento demais e buscar otimizações (ou descartar approach).
- **EXP-04: Extensão Longa Duração** – *Hipótese:* O Broadcast Extension permanece ativo por horas sem ser encerrado pelo iOS.
 - *Implementação:* Iniciar broadcast num app de teste com fundo estático (por exemplo, uma tela branca) e deixar rodando por 2h.
 - *Métrica:* Se o callback `broadcastFinished()` é invocado ou se a extensão cessa. Monitorar uso de CPU/mem.
 - *Sucesso:* Fica ativa sem erro por 2 horas.
 - *Abandono:* Se for finalizada antes de, digamos, 30 minutos ou entrar em crash, considerar estratégia de cronômetro para reiniciar ou replanejar abordagens (isso indica instabilidade).

- **EXP-05: Comunicação ext→app** – *Hipótese*: Extensão pode notificar o app principal em tempo real.
 - *Implementação*: Na extensão, após OCR e parse, escrever resultado no UserDefaults do App Group e enviar `CFNotificationCenterPostNotification` (Darwin) com nome personalizado. No app principal, registrar observador do Darwin: ao receber, ler UserDefaults e atualizar UI mínima (p.ex. label no debug).
 - *Métrica*: Tempo de ida e volta (ext → notificação → app).
 - *Sucesso*: App principal atualiza interface (ou Live Activity) <100ms após a notificação.
 - *Abandono*: Se não conseguir comunicação (ex. notificação não pega), avaliar outras formas (p.ex. URL scheme, mas nenhuma solução robusta).
- **EXP-06: Live Activity atualização** – *Hipótese*: Uma Live Activity pode ser atualizada com baixa latência a partir de dados locais.
 - *Implementação*: Simule envio de atualização via `Activity.update(using:)` com dados de teste a cada 5 segundos. Meça visualmente ou via logs em quanto tempo aparece.
 - *Métrica*: Latência de atualização (tempo entre update e tela).
 - *Sucesso*: Atualização refletida ~instantaneamente (<500ms).
 - *Abandono*: Se for muito lenta ou imprevisível, reavaliar se Live Activities é adequado.
- **EXP-07: Consumo de Bateria** – *Hipótese*: Uso contínuo de Broadcast+OCR não drena a bateria excessivamente.
 - *Implementação*: Disparar EXP-04 em um iPhone totalmente carregado. Roda por 1h em background (com OCR ativo, talvez alternando tela para simular oferta). Registrar consumo de bateria (porcentagem/minuto) e temperatura.
 - *Métrica*: % de bateria por hora, elevação de temperatura.
 - *Sucesso*: Menos de ~15%/hora (variável). Acima disso, seria cansativo para motorista.
 - *Abandono*: Se >25%/hora ou superaquecimento (ex. CPU perto de 100%). Isso indicaria risco de desconforto e limitações termos de experiência.
- **EXP-08: 99 funciona?** – *Hipótese*: O mesmo pipeline OCR captura as ofertas da 99.
 - *Implementação*: Repetir EXP-01 e EXP-02 abrindo o app 99 (Android) ou testando tela equivalente (se possível side-load ou simulação).
 - *Métrica*: Textos da 99 (preço, km, etc.) reconhecidos no log.
 - *Sucesso*: OCR detecta valores corretos da 99.
 - *Abandono*: Se interface gráfica da 99 for muito diferente (p.ex. fundos complexos) e o OCR falhar, talvez seja necessário ajustar regionOfInterest ou até conclusão de inviabilidade parcial.
- **EXP-09: inDrive funciona?** – *Hipótese*: Pipeline funciona no inDrive.
 - *Implementação*: Similar ao EXP-08 para o app inDrive.
 - *Métrica*: Reconhecimento de textos de oferta do inDrive.
 - *Sucesso*: Captura e parsing corretos de inDrive.

- **Abandono:** Se inDrive usar UI dinâmica ou vetores que o OCR não reconhece (por exemplo, OpenGL), ou se o app bloquear capturas, pode ser inviável capturar dados. Nesse caso, priorizar Uber/99.

Cada experimento deve ser documentado: se o critério de sucesso for atingido, avançamos; se o critério de aborto ocorrer, interrompemos ou mudamos radicalmente a abordagem.

S. Plano de Migração do Android

Para migrar progressivamente o app Android atual para uma arquitetura compartilhada:

1. **Isolamento do domínio existente:** No app Android atual (em Kotlin), identifique todo o código de lógica de negócio, cálculos e modelos (por exemplo, classes `Offer`, `CalcMotEngine`, rules do semáforo). Crie um módulo comum (KMP) e mova esses arquivos para `shared/commonMain`. Ajuste dependências (por exemplo, remova referências a Android SDK nas classes de domínio).
2. **Definir APIs comuns:** Crie interfaces (`expect / actual` ou plain interfaces em common) para elementos de plataforma. Exemplo: `interface TextRecognizer` (common) e implementações em Android (`actual class TextRecognizer { ... }` usando ML Kit) e iOS (`actual class VisionRecognizer { ... }`). Para parsers, já aproveite lógica comum de manipulação de strings no shared.
3. **Configurar o Gradle Multiplatform:** Converter o projeto Gradle para KMP com targets `android()` e `ios()`. Exporte um framework iOS ao buildar (`iosX64`, `iosArm64`). Certifique-se de que o código compartilhado compile sem erros em ambos os targets.
4. **Ajustar infraestrutura Android:** No módulo `androidApp`, mantenha o `AccessibilityService` e o overlay Android atuais. Altere-os para usar as classes do módulo compartilhado para todos os cálculos. Por exemplo, em vez de calcular `pricePerKm` dentro da activity, chame `CalcMotEngine().analyze(offer)` do shared. Deixe o ML Kit / OCR do Android como implementação de `TextRecognizer` no `shared/androidMain`.
5. **Desenvolver a versão iOS:** Crie o projeto Xcode (`iosApp`), importe o framework compartilhado. Implemente as partes iOS: a UI (SwiftUI/ActivityKit), o `Broadcast Upload Extension`, e use Vision para OCR. Nos pontos de lógica (parser, cálculos), chame o código Kotlin compartilhado (`CalcMotEngine`, etc).
6. **Teste cruzado:** Garanta que tanto Android quanto iOS usem o mesmo core de cálculos. Escreva testes unitários no `shared/commonTest` para validar resultados idênticos em ambos.
7. **Implantação incremental:** Antes de liberar no Android existente, uma *pre-release* (beta) com o código refatorado para KMP deve passar sem quebrar funcionalidade. Depois, foque no iOS.

Esse plano permite reuso máximo do código de domínio, minimiza divergências entre plataformas e mantém o Android funcionando enquanto o iOS é construído.

T. Roadmap de Implementação

1. **Fase 0 – Viabilidade:**
2. Conduzir EXP-01 a EXP-04 (POC ReplayKit, OCR, latência, duração).
3. Se passar, validamos que o approach básico funciona. Caso contrário, interromper.
4. Duração: 2 semanas.

5. Fase 1 – Core compartilhado:

- 6. Criar módulo KMP (`shared`) com domínio, cálculos e parsers.
- 7. Refatorar parte do Android (separar ofertas e classificações).
- 8. Configurar projetos multiplataforma (Gradle).
- 9. Duração: 3 semanas.

10. Fase 2 – Captura iOS:

- 11. Desenvolver o *Broadcast Upload Extension* em Swift.
- 12. Integrar `RPSystemBroadcastPickerView` na UI de teste.
- 13. Implementar OCR básico (sem parse) para testar frames.
- 14. Executar EXP-01, EXP-02 em iPhone reais.
- 15. Duração: 4 semanas.

16. Fase 3 – Parsers e OCR completos:

- 17. Finalizar a lógica de OCR (`regionOfInterest`, `customWords`).
- 18. Implementar parsers do Uber/99/inDrive no módulo compartilhado (baseados no texto que o extension enviará).
- 19. Conectar extensão com o módulo KMP para construir `Offer` e `OfferAnalysis`.
- 20. Executar EXP-08 e EXP-09 (99, inDrive).
- 21. Duração: 4 semanas.

22. Fase 4 – Apresentação:

- 23. Construir a interface final: Live Activities (com SwiftUI+ActivityKit) ou notificação personalizada.
- 24. Integrar a comunicação ext→app (App Group + Darwin).
- 25. Testar EXP-05 (comunicação) e EXP-06 (Live Activity).
- 26. Refinar UX: som, vibração se for o caso.
- 27. Duração: 3 semanas.

28. Fase 5 – Testes de campo e otimização:

- 29. Realizar testes de usuários (motoristas) com protótipo, coletar feedback.
- 30. Medir consumo de bateria e performance (EXP-07).
- 31. Ajustar frame skipping, latência, melhorias de OCR conforme necessário.
- 32. Duração: 2-3 semanas.

33. Fase 6 – App Store:

- 34. Preparar recursos (ex: explicações de privacidade, imagens de screenshot da IU iOS).
- 35. Submeter à Apple com documentação explicando caso de uso.
- 36. Reagir a eventuais reprovações (logs, clarificações).
- 37. Lançamento global após aprovação.

38. Duração: 2 semanas (revisão + ajustes).

Cada fase termina com critérios de aceitação: os EXPs relevantes devem passar para prosseguir.

U. Critérios de Descarte (Kill Criteria)

Devemos definir claramente quando **abandonar** uma abordagem:

- **Frames não chegam (EXP-01 falha):** Se o Broadcast Extension não capturar *nenhum* frame do app alvo (por exemplo, por falha de configuração ou proibição inesperada), interromper imediatamente o plano de ReplayKit. Buscar alternativas menores (ex: compartilhamento manual).
- **OCR inaceitável (EXP-02 falha):** Se, mesmo após otimizações, o Vision OCR não conseguir extrair corretamente os valores (ex.: falhar em >50% das ofertas de teste), considerar a abordagem inviável. Talvez não haja solução sobrenatural além de API oficial do parceiro (que não existe). Nesse caso, o projeto CalcMot iOS precisa repensar funcionalidades (ex.: só dar feedback qualitativo).
- **Alta latência (EXP-03 ruim):** Se o pipeline completo (do frame à análise) for consistentemente >1 segundo nos dispositivos alvo, a experiência é muito lenta. Um limite razoável seria ~500ms; acima de 1s pode inviabilizar o produto. Se não conseguir otimizar abaixo disso, reavaliar as features (menos OCR, mais heurística, etc.) ou até abandonar.
- **Extensão encerrada (EXP-04 falha):** Se o Broadcast Extension cair de forma confiável (ex.: após poucos minutos) sob condições normais de uso, não podemos depender dele. Nesse caso, a arquitetura fundamental falha e devemos parar.
- **Falha de comunicação (EXP-05 falha):** Se não conseguirmos notificar o app principal após 1–2 tentativas (por problemas de App Groups/Darwin), nossa capacidade de apresentar dados em tempo real fica prejudicada. Se soluções alternas (por exemplo, push interno) não resolverem, precisaríamos replanejar a apresentação (talvez só local notificações).
- **Atualização Live Activity falha (EXP-06 falha):** Se a Live Activity não puder ser atualizada de forma confiável (p.ex. se iOS 16 ainda tiver muitas limitações ou bugs), teremos que confiar somente em notificações tradicionais (menos ideal). Não é fim-de-linha, mas reduz valor percebido.
- **Bateria/thermal excessivo (EXP-07 falha):** Se nos testes a bateria derrete >30% por hora ou o aparelho esquenta perigosamente, isso torna o app impraticável a longo prazo. Nesses casos, a funcionalidade deve ser reduzida ou suspensa.
- **Rejeição da Apple:** Se em diálogo com App Review ficarmos claros que a política atual não permite o uso pretendido (mesmo com consentimento do usuário), isso constituiria crítico. Por exemplo, se recebemos reprovação direta (Category 2.5 ou 5.1) sem chance de recurso, teríamos que abandonar a abordagem – possivelmente limitando o app ao Android ou pensando em algum backend/autorização da própria Uber/99.

Estes critérios serão monitorados em cada etapa experimental. Se um critério de abandono for atendido, interromperemos o desenvolvimento nessa direção e comunicaremos os próximos passos alternativos.